Biweekly Assessment BW05 — Weeks 9–10

Sessions covered: S41–50
Topics: `try` / `catch` / `finally`, try-with-resources, `throw` + custom exceptions; `Path` / `Files` UTF-8 write/append/read, simple CSV; Git add/commit/branch/push, Java/Maven `.gitignore`, README (no secrets); MySQL setup/connect, table/row vs Excel, DDL+PK, DML, DQL `WHERE` / `ORDER BY` / `LIMIT`, ER sketch

Timebox: 45–60 minutes (homework or Friday of the even week).
Rules: Closed notes unless your trainer allows open notes. No copying.
Coding constraint: `try` / `catch` / `throw` / try-with-resources and `java.nio.file` (`Path` / `Files`, UTF-8, simple CSV) are OK. **No JDBC** (`Connection` / `PreparedStatement` code). **No JOIN SQL** in coding. No Spring. No Lombok. Logging is not required.

A. Multiple Choice (12)

Circle the best option.

1. A `finally` block runs:
   a) After `try` / `catch`, whether you succeeded, caught, or `return`ed
   b) Only on success
   c) Only on error
   d) Never if you wrote `catch`
2. `try (Scanner sc = new Scanner(source)) { ... }` mainly:
   a) Auto-closes the `Scanner` (`AutoCloseable`)
   b) Skips UTF-8
   c) Needs Spring
   d) Replaces `PRIMARY KEY`
3. A custom exception class usually:
   a) `extends Exception` or `extends RuntimeException`, and you `throw new ...`
   b) Must `extends String`
   c) Must be a Spring bean
   d) Cannot carry a message
4. `Path.of("notes.txt")` names a file. `Files.writeString(path, text, UTF_8)` with **no** extra options:
   a) Creates or **replaces** the file (old content is gone)
   b) Always appends
   c) Needs a MySQL connection
   d) Ignores UTF-8 on purpose
5. `git add` then `git commit`:
   a) Stages then records a snapshot
   b) Deploys to AWS
   c) Creates a MySQL table
   d) Runs `javac` only
6. `git push` sends commits to:
   a) A remote (e.g. GitHub)
   b) MySQL `training_db`
   c) Only `*.class` files
   d) The JDK installer

7. A Java / Maven `.gitignore` should **not** track:
   a) `*.class`, `target/`, `.env` (secrets stay out of Git)
   b) `README.md` and `.java` sources
   c) Only `pom.xml`
   d) The `.gitignore` file itself forever
8. `CREATE TABLE products (product_id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(80), price INT);` is:
   a) DDL — defines structure; PRIMARY KEY uniquely identifies a row
   b) DML only (`INSERT`)
   c) A Git commit
   d) try-with-resources
9. `DELETE FROM products;` with **no** `WHERE`:
   a) Deletes **every** row — always use `WHERE`; test with `SELECT` first
   b) Deletes only the first row
   c) Is DDL (`DROP TABLE`)
   d) Pushes to GitHub
10. `SELECT name, price FROM products WHERE price < 100 ORDER BY price ASC LIMIT 5` is:
    a) DQL — filter, sort, and cap rows (does not change data)
    b) DDL — `CREATE TABLE`
    c) TCL — `COMMIT`
    d) A Git merge
11. JOIN is used to:
    a) Combine related tables (a foreign key matches a primary key)
    b) Commit git
    c) Close a `Scanner`
    d) Style HTML
12. When Java later sends SQL, pass user data with `?` placeholders, not string concatenation, because:
    a) Concatenating quotes into SQL lets attackers inject commands (SQL injection)
    b) `?` is a Git branch
    c) MySQL cannot store strings
    d) It replaces `CREATE TABLE`

B. Coding (2)

1. Write `CanteenPay` with a private `balance` (INR, start at `100`) and `pay(int amount)`. `pay` **throws** `IllegalArgumentException` if `amount <= 0`, and **throws** custom `InsufficientFundsException` (`extends Exception`) if `amount > balance`. On success, subtract and print `Paid ₹` + amount + `, left ₹` + balance. In `main`, try `pay(40)`, then `pay(80)`, then `pay(-5)` — catch each type with a clear message. No files. No JDBC.

```
// your code
```

2. Write `CanteenCsv` for **Employee Management Portal** expenses. Use `Path.of("canteen_expenses.csv")` and UTF-8. If the file is missing, write header `item,amount`. Method `appendRow(item, amount)` **throws** `IllegalArgumentException` if `amount <= 0`; otherwise **append** `item,amount` with `CREATE` + `APPEND`. In `main`: append `Tea` at `45`, `Bus` at `30`, then try amount `0` (catch and print). Then `Files.readString` and print the file. Catch `IOException`. No JDBC. No Git.

```
// your code
```

C. Short answer (1)

In 3–6 sentences, explain a **campus library** ER sketch (or **Swiggy** Customer / Product / Order): which boxes you draw, which 1:N line, why a **PRIMARY KEY**, and why this is safer than one Excel sheet. Do **not** write JOIN SQL.

(Write 3–6 sentences.)